LAB # 11

Implement deadlocks detection in a system using wait-for graphs by takinginput for processes and their allocated resources, enhancing process management and system reliability in operating systems.

Task1:

Given the resource allocation graph below. Write the code to detect the Deadlock in the system using cyclic feature of the graph.

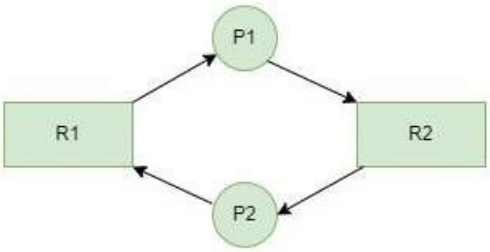
//Data structure for the problem below

//Request Matrix

Int request[2][2];

//Allocation Matrix

Int allocation[2][2];



Code:

```
#include <stdio.h>
int visited[2], recStack[2];
int waitFor[2][2];
int isCycle(int v)
{
    if (recStack[v])
        return 1;
    if (visited[v])
        return 0;
    visited[v] = 1;
    recStack[v] = 1;

    for (int i = 0; i < 2; i++)
    {
        if (waitFor[v][i])
        {
            if (isCycle(i))
                return 1;
        }
    }
    recStack[v] = 0;
    return 0;
}
int main()
{
    int request[2][2] = {
        {0, 1},
 {1, 0}
    };
```

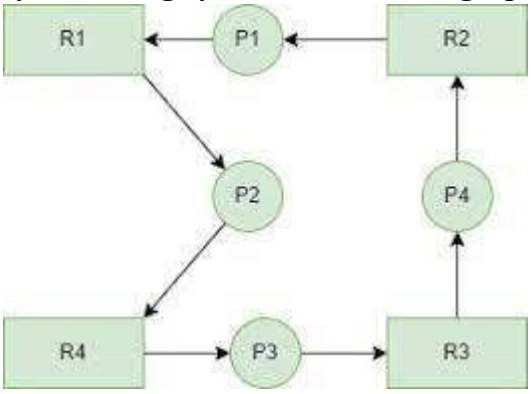
```
int allocation[2][2] = {
        {1, 0},
        {0, 1}
    };
    // Build Wait-For Graph
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            waitFor[i][j] = 0;
    for (int i = 0; i < 2; i++)
        for (int r = 0; r < 2; r++)
            if (request[i][r])
                for (int p = 0; p < 2; p++)
                    if (allocation[p][r])
                        waitFor[i][p] = 1;
    for (int i = 0; i < 2; i++)
    {
        if (isCycle(i))
        {
            printf("Deadlock Detected\n");
            return 0;
        }
    }
    printf("No Deadlock\n");
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab11$ ./Task1.out
Deadlock Detected
```

Task2:

Given the resource allocation graph below. Write the code to detect the Deadlock in the system using cyclic feature of the graph.



//Data structure for the problem below

//Request

Matrix Int

request[4][4];

//Allocation Matrix

Int allocation[4]

[4];

Code:

```
#include <stdio.h>

int visited[4], recStack[4];
int waitFor[4][4];
int isCycle(int v)
{
    if (recStack[v])
        return 1;

    if (visited[v])
        return 0;

    visited[v] = 1;
    recStack[v] = 1;

    for (int i = 0; i < 4; i++)
        if (waitFor[v][i] && isCycle(i))
            return 1;

    recStack[v] = 0;
    return 0;
}

int main()
{
    int request[4][4] = {
        {0,1,0,0},
        {0,0,1,0},
        {0,0,0,1},
        {1,0,0,0}
    };
    int allocation[4][4] = {
        {1,0,0,0},
        {0,1,0,0},
        {0,0,1,0},
        {0,0,0,1}
    };
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            waitFor[i][j] = 0;

    for (int i = 0; i < 4; i++)
        for (int r = 0; r < 4; r++)
            if (request[i][r])
                for (int p = 0; p < 4; p++)
                    if (allocation[p][r])
                        waitFor[i][p] = 1;
```

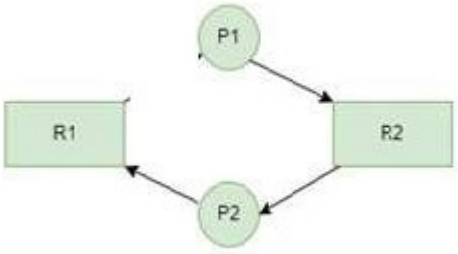
```
    for (int i = 0; i < 4; i++)
        if (isCycle(i))
        {
            printf("Deadlock Detected\n");
            return 0;
        }

    printf("No Deadlock\n");
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab11$ ./Task2.out
Deadlock Detected
```

Task 3: (No Deadlock Scenario)

Given the resource allocation graph below. Write the program to detect the Deadlock in the system using cyclic feature of the graph.



Code:

```
#include <stdio.h>

int visited[3], recStack[3];
int waitFor[3][3];

int isCycle(int v)
{
    if (recStack[v])
        return 1;
    if (visited[v])
        return 0;

    visited[v] = 1;
    recStack[v] = 1;

    for (int i = 0; i < 3; i++)
        if (waitFor[v][i] && isCycle(i))
            return 1;

    recStack[v] = 0;
```

```
    return 0;
}

int main()
{
    int request[3][3] = {
        {0,1,0},
        {0,0,1},
        {0,0,0}
    };

    int allocation[3][3] = {
        {1,0,0},
        {0,1,0},
        {0,0,1}
    };

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            waitFor[i][j] = 0;

    for (int i = 0; i < 3; i++)
        for (int r = 0; r < 3; r++)
            if (request[i][r])
                for (int p = 0; p < 3; p++)
                    if (allocation[p][r])
                        waitFor[i][p] = 1;

    for (int i = 0; i < 3; i++)
        if (isCycle(i))
        {
            printf("Deadlock Detected\n");
            return 0;
        }

    printf("No Deadlock\n");
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab11$./Task3.out
No Deadlock
```

Task 4:

Write the generic program to detect the deadlock in the system with the variable number of processes and resources.

Steps to create a Resource allocation graph.

1. Prompt the user for the number of processes.
2. Prompt the user for the number of resources (with a single instance)
3. Prompt for resource assignment to processes. (in case of no assignment assign zero to a resource)
4. Prompt for request to resource by processes. (in case of no request assign zero to process)
5. Convert the resource allocation to wait for graph and detect deadlock(cycle)
6. Print the number of cycles.
7. Print the processes and resources which are creating deadlock in each cycle.

Code:

```
#include <stdio.h>

int n;
int waitFor[20][20];
int visited[20], recStack[20];
int cycleCount = 0;
int isCycle(int v)
{
    if (recStack[v])
        return 1;
    if (visited[v])
        return 0;
    visited[v] = 1;
    recStack[v] = 1;

    for (int i = 0; i < n; i++)
        if (waitFor[v][i] && isCycle(i))
            return 1;
    recStack[v] = 0;
    return 0;
}
int main()
{
    int p, r;
    printf("Enter number of processes: ");
    scanf("%d", &p);
    printf("Enter number of resources: ");
    scanf("%d", &r);

    n = p;
    int request[p][r], allocation[p][r];
    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < p; i++)
        for (int j = 0; j < r; j++)
```

```
            scanf("%d", &allocation[i][j]);
    printf("Enter Request Matrix:\n");
    for (int i = 0; i < p; i++)
        for (int j = 0; j < r; j++)
            scanf("%d", &request[i][j]);
     for (int i = 0; i < p; i++)
        for (int j = 0; j < p; j++)
            waitFor[i][j] = 0;
    for (int i = 0; i < p; i++)
        for (int k = 0; k < r; k++)
            if (request[i][k])
                for (int j = 0; j < p; j++)
                    if (allocation[j][k])
                        waitFor[i][j] = 1;

    for (int i = 0; i < p; i++)
        if (isCycle(i))
            cycleCount++;
    if (cycleCount)
        printf("Deadlock Detected\n");
    else
        printf("No Deadlock\n");

    printf("Number of Cycles: %d\n", cycleCount);
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab11$ ./Task4.out
Enter number of processes: 4
Enter number of resources: 4
Enter Allocation Matrix:
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
Enter Request Matrix:
0 1 0 0
0 0 1 0
0 0 0 1
1 0 0 0
Deadlock Detected
Number of Cycles: 4
```